

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

Факультет «Информатика и системы управления»
Кафедра «Компьютерные системы и сети»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «Проектная деятельность»

**ГЕНЕРАЦИЯ ОПИСАНИЙ ОБЪЯВЛЕНИЙ АВИТО
ПО ФОТОГРАФИИ И ПАРАМЕТРАМ ТОВАРА**

Студент группы ИУ6-74Б
Хаялиев Раниль Рустемович
_____ Р. Р. Хаялиев

Руководитель курсовой работы
_____ М. А. Скворцова

Москва 2026

РЕФЕРАТ

Отчет 34 с., 5 рис., 9 табл., 12 источн., 1 прил.

ГЕНЕРАЦИЯ ТЕКСТА, ОБЪЯВЛЕНИЕ, АВИТО, МУЛЬТИМОДАЛЬНАЯ МОДЕЛЬ, QWEN, ДООБУЧЕНИЕ, ОЦЕНКА КАЧЕСТВА, ВЕБ-СЕРВИС

Объект разработки — система подготовки описаний товарных объявлений по фотографии, заголовку, категории и структурированным характеристикам. Цель работы — исследовать способы генерации русскоязычного текста и реализовать демонстрационный сервис.

Использованы готовые данные Авито, базовые и дообученные языковые и мультимодальные модели, архитектура CLIP с ruGPT, программные проверки и оценка языковыми моделями. Реализован MVP на React, FastAPI и vLLM. Проверены состав выборок и сохраненные ответы генераторов.

Базовая VLM прошла текущую проверку формата в 86 случаях из 100, SFT VLM — в 18, CLIP с ruGPT — ни в одном. Эти результаты характеризуют соблюдение формальных ограничений. Оценки качества из презентации рассматриваются отдельно. Решение пригодно для демонстрации и дальнейших исследований. Промышленное внедрение требует согласования протокола оценки и проверки на расширенной выборке.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	5
ВВЕДЕНИЕ.....	6
1 Анализ задачи и требований.....	7
1.1 Проблема и практическая значимость.....	7
1.2 Аналоги и выбор общего подхода.....	7
1.3 Требования и границы MVP.....	8
2 Исходные данные.....	10
2.1 Состав и контракт данных.....	10
2.2 Проверка размеров и разделения выборок.....	11
2.3 Распределение товарных категорий.....	12
2.4 Происхождение характеристик и качество целей.....	13
3 Методы генерации описаний.....	14
3.1 Постановка мультимодальной генерации.....	14
3.2 Базовые языковая и мультимодальная модели.....	15
3.3 Дообучение с низкоранговыми адаптерами.....	16
3.4 Архитектура CLIP и ruGPT.....	17
3.5 Выбор модели для демонстрационного сервиса.....	18
4 Реализация программной системы.....	19
4.1 Компоненты и взаимодействие.....	19
4.2 Валидация и обработка ошибок.....	20
4.3 Пользовательский интерфейс.....	21
4.4 Развертывание и программные проверки.....	22
5 Методика и результаты оценки.....	23

5.1 Двухэтапный контроль качества.....	23
5.2 Ансамбль судей и сопоставимость результатов.....	24
5.3 Сравнение подходов по материалам проекта.....	24
5.4 Повторная проверка сохраненных ответов.....	26
5.5 Анализ конкретного ответа и ограничений.....	27
6 Эксплуатационные характеристики и развитие.....	28
6.1 Производительность и границы нагрузки.....	28
6.2 Практический эффект и дальнейшая работа.....	29
6.3 План развития архитектуры.....	30
ЗАКЛЮЧЕНИЕ.....	31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	32
ПРИЛОЖЕНИЕ А (справочное) Порядок запуска и фиксации эксперимента.....	34

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяют следующие сокращения и обозначения.

API — программный интерфейс взаимодействия компонентов

CLIP — модель сопоставления изображений и текста

CSV — текстовый формат табличных данных

DPO — прямая оптимизация модели на предпочтениях

GPU — графический процессор

JSON — формат представления структурированных данных

LLM — большая языковая модель

LoRA — адаптация модели с помощью матриц малого ранга

MVP — минимально жизнеспособный продукт

QLoRA — дообучение с низкоранговыми адаптерами и квантованной базовой моделью

SFT — дообучение на примерах с целевыми ответами

VLM — модель, обрабатывающая изображения и текст

VRAM — видеопамять графического процессора

p95 — значение задержки, не превышаемое в 95 % измерений

ВВЕДЕНИЕ

При размещении товарного объявления продавец должен превратить сведения о предмете в связный и понятный текст. Название и отдельные характеристики часто уже введены, однако описание приходится составлять вручную. Из-за этого полезные свойства пропускаются, а содержание заменяется общими фразами. Автоматическая генерация может сократить работу продавца, если сохраняет факты и не добавляет неподтвержденные обещания.

Цель курсовой работы — разработать прототип системы генерации описаний объявлений Авито по одной фотографии и параметрам товара и исследовать качество нескольких способов решения. Для достижения цели рассмотрены предметная область и аналоги, проверены готовые выборки, изучены базовые и дообученные модели, реализованы сервис и процедура оценки.

Объект исследования — подготовка текста товарного объявления. Предмет исследования — объединение структурированных характеристик и визуальной информации при генерации русскоязычного описания. Практический результат — локальный веб-сервис, в котором пользователь задает сведения о товаре и получает текст для последующего использования.

Исходные данные предоставлены как подготовленные таблицы Авито. Получение исходной выгрузки и повторная промышленная очистка не входят в границы разработки. В отчете проверяются фактически доступные файлы и объясняется, как их свойства влияют на обучение и оценивание. Основой служат презентация проекта [1], исходный код и ноутбуки [2], модуль LLM-as-a-Judge [3], а также предшествующее обоснование проекта [4].

Работа выполнена в рамках совместного проекта с Даниилом Ханенко. Настоящий индивидуальный отчет подготовлен Хаялиевым Ранилем Рустемовичем. Распределение конкретных программных модулей между участниками исходными материалами не устанавливается. Структура и оформление отчета приняты по ГОСТ 7.32–2017 [5].

1 Анализ задачи и требований

1.1 Проблема и практическая значимость

Описание объявления связывает изображение товара с характеристиками, важными для решения о покупке. Фотография показывает внешний вид, но обычно не позволяет достоверно определить объем памяти, точную модель, историю эксплуатации или комплект поставки. Структурированные поля дополняют изображение, а текст объединяет эти сведения в удобную форму.

Недостаток информации увеличивает число уточняющих действий. Покупателю приходится писать продавцу, сопоставлять заголовок с фотографией и самостоятельно искать нужные параметры. Для продавца это означает дополнительные сообщения и затраты времени. Поэтому ценность генератора состоит в подготовке понятного черновика с подтвержденными характеристиками.

Промышленная применимость задачи показана в публикации команды Авито: после внедрения генерации в категории одежды и обуви 60 % опрошенных продавцов положительно оценили тексты, а число заказов с доставкой выросло на 1,7 % [6]. Это результат чужого А/В-теста, подтверждающий актуальность направления, но не ожидаемый эффект данного прототипа.

В учебном проекте успех определяется на двух уровнях. На техническом уровне сервис должен принять данные и вернуть связный текст. На содержательном уровне текст должен сохранить существенные признаки товара, соблюдать ограничения и не вводить покупателя в заблуждение. Работоспособность интерфейса сама по себе не доказывает качество модели.

Наиболее существенны ошибки чисел, моделей и состояния товара. Замена ширины ремешка с 26 на 25 мм меняет совместимость, даже если остальной текст грамматически корректен. По этой причине приоритет отдан достоверности свойств, а выразительность и разнообразие текста рассматриваются после выполнения этого требования.

1.2 Аналоги и выбор общего подхода

Простейший подход — подстановка характеристик в шаблон. Он обеспечивает предсказуемую структуру и удобен для узкого ассортимента. Однако

шаблон самостоятельно не извлекает признаки изображения, требует согласования русских слов и плохо переносится на категории с разными наборами полей. Его целесообразно сохранять как контрольный вариант и возможный резервный режим.

Языковая модель обобщает текстовые характеристики и формирует связное описание. При отсутствии фотографии она ограничена заголовком и параметрами. Мультимодальная модель дополнительно учитывает цвет, форму и заметные особенности предмета, но может ошибаться при интерпретации изображения. Сравнение подходов представлено в таблице 1.

Таблица 1 — Сравнение способов генерации

Подход	Входное фото	Основное ограничение
Шаблон	Нет	Поддержка правил для категорий
Текстовая LLM	Нет	Нет новых визуальных признаков
CLIP и ruGPT	Да	Требуется обучение связи модальностей
Готовая VLM	Да	Нужен контроль фактических ошибок
SFT VLM	Да	Зависимость от целевых текстов

В последующей публикации Авито описаны переход к нескольким изображениям и переразметка обучающих описаний. Авторы сообщили о росте числа генераций на 5,7 % и более частом получении подходящего текста с первой попытки на 10,8 % [7]. Этот опыт поддерживает гипотезу о важности качества целевых ответов.

Для MVP выбрана готовая Qwen3-VL-4B-Instruct, поскольку она принимает изображение и текст в одном запросе. Карточка модели описывает мультимодальный интерфейс и способы локального запуска [8]. Альтернативные архитектуры сохранены в исследовательской части для сравнения, а не объявлены эквивалентными промышленному решению Авито.

1.3 Требования и границы MVP

Пользователь загружает главное изображение, вводит заголовок и категорию, при необходимости передает характеристики в JSON. Сервис формирует описание на русском языке. Пользователь остается ответственным за

проверку результата перед публикацией: прототип не размещает объявление на площадке и не управляет сделкой.

Функциональные требования задают прием изображения JPEG, PNG или WEBP, проверку обязательных текстовых полей, разбор JSON-объекта и возврат результата через HTTP API. Интерфейс должен показывать ожидание, ошибки и возможность копирования текста. Признак доступности API отделен от готовности модели к генерации.

Содержательный контракт предусматривает один абзац длиной не более 70 слов. Описание должно начинаться с предмета продажи, включать подтвержденные характеристики и использовать не более одного-двух значимых визуальных признаков. Запрещены контакты, ссылки, цена, доставка, гарантии, приветствия, разметка и объяснение того, как был получен ответ [2, 3].

Фотография не является достаточным основанием для вывода об исправности электроники или полном комплекте автозапчасти. Для животных нельзя выводить здоровье, характер и прививки из внешнего вида. Если характеристика отсутствует, правильным поведением считается ее пропуск, даже если итоговый текст оказывается коротким.

Нефункциональные требования связаны с воспроизводимостью запуска и умеренной задержкой ответа. Для демонстрации используется локальная GPU-конфигурация и контейнерное развертывание. Авторизация, история генераций, промышленная модерация и автоматическая публикация в состав MVP не включены.

Критерии приемки должны проверяться отдельно: успешность HTTP-запроса, соблюдение формата, достоверность фактов и пригодность текста для продавца. Такое разделение позволяет локализовать причину неудачи и не подменять качество содержания доступностью сервиса.

2 Исходные данные

2.1 Состав и контракт данных

В проекте предполагается, что компания Авито предоставила данные в пригодном для обработки формате. Поэтому рабочий вход рассматривается как готовая таблица объявлений, связанная с изображениями через `image_id`. Описание исходной подготовки в презентации используется как контекст происхождения данных, но не заменяет проверку доступных файлов.

Таблицы содержат заголовок, исходное описание продавца, категорию, микрокатегорию, цену, сведения о продавце и показатель `quality_score`. Для генератора используются только поля, предусмотренные его контрактом. Цена и служебные признаки отбора не должны автоматически попадать в текст. Основные поля перечислены в таблице 2.

Таблица 2 — Назначение полей набора данных

Поле	Роль в проекте
<code>item_id</code>	Идентификатор объявления и контроль пересечений
<code>image_id</code>	Связь с главной фотографией
<code>title</code>	Наименование продаваемого объекта
<code>category_name</code>	Контекст товарной категории
<code>params</code>	JSON-объект подтвержденных характеристик
<code>description</code>	Исходный текст продавца для обучения и анализа
<code>quality_score</code>	Служебная оценка отбора, не вход генератора

Словарь `params` зависит от категории: для электроники важны модель и память, для одежды — размер и материал. Полная унификация ключей не требуется для построения запроса, однако единицы измерения и смысл значений должны сохраняться.

В локальной поставке присутствуют CSV, но отсутствует полный каталог фотографий. Поэтому повторно проверены табличный контракт и сохраненные тексты; полный мультимодальный прогон по исходным изображениям при

подготовке отчета не выполнялся. Это ограничение не меняет схему применения данных в самой системе.

2.2 Проверка размеров и разделения выборок

Файлы прочитаны с учетом UTF-8 BOM, чтобы первый столбец распознавался как `item_id`. Подсчет записей дал 30 000 строк в `train`, 100 в `valid` и 2 937 в дополнительной выборке, имя которой содержит обозначение `Зк`. Последнее является округленным названием, а не точным размером файла. Сводка приведена в таблице 3.

Таблица 3 — Результат проверки локальных выборок

Выборка	Строк	Пустые params	Уникальные item_id
train	30 000	6	30 000
valid	100	0	100
Дополнительная Зк	2 937	11	2 937

Во всех строках `params` разбирается как JSON-объект. Под пустыми параметрами здесь понимается отсутствие содержательных полей, в частности пустой словарь. Корректный синтаксис не доказывает истинность каждого значения, но позволяет надежно передавать характеристики между компонентами.

Обучающая и проверочная выборки не пересекаются по `item_id`. Дополнительная выборка также не пересекается с `valid`. Идентификаторы 100 сохраненных ответов базовой VLM совпадают с идентификаторами проверочного набора. Такая проверка исключает очевидное смешение объектов на уровне идентификаторов, но не обнаруживает разные объявления с одинаковыми фотографиями или почти совпадающими текстами.

Для более строгого контроля требуется поиск дубликатов изображений и близких описаний, а также группировка по продавцам. Два объявления одного продавца могут использовать общий рекламный шаблон и создавать зависимость между обучением и проверкой. В доступных материалах отдельный независимый тестовый набор не выделен.

Выборка `valid` предназначена для сопоставления подходов. Если по ней многократно выбирать промпт, параметры и контрольную точку, результат

постепенно подстраивается под эти 100 объектов. Поэтому окончательную оценку будущей версии следует выполнять на новом наборе, который не участвовал в разработке.

2.3 Распределение товарных категорий

Состав обучающей выборки существенно неравномерен. Доли рассчитаны по фактическому train, а не перенесены из диаграммы исходной выгрузки. На рисунке 1 показано распределение 30 000 записей по восьми категориям.



Рисунок 1 — Распределение категорий в обучающей выборке

Транспорт представлен 12 885 записями, личные вещи — 5 920, товары для дома и дачи — 5 529. Вместе эти три категории дают 81,1 % train. Животные представлены только 153 записями, то есть 0,51 %. Поэтому общее среднее качество преимущественно отражает поведение на наиболее частых товарах.

В valid находятся 43 транспортных объявления, 20 объявлений о личных вещах, 18 — для дома и дачи, 7 — об электронике, 6 — о хобби и отдыхе, по 3 — о недвижимости и готовом бизнесе с оборудованием. Животных в проверочном наборе нет. Следовательно, результаты этого прогона не дают эмпирического основания оценить работу на животных.

При расширении проверки следует совместить два набора: пропорциональный реальному потоку и сбалансированный диагностический. Первый показывает ожидаемое качество на типичной нагрузке, второй выявляет слабые категории. Для редких групп полезно отдельно учитывать ошибки состояния, размеров и неподтвержденных характеристик.

2.4 Происхождение характеристик и качество целей

В презентации подготовка данных описана как последовательность фильтров наличия фотографии и описания, длины текста, положительной цены, отсутствия контактов, уникальности, времени размещения и товарной категории. Для отбора используется `quality_score`, учитывающий свойства текста, продавца и объявления [1]. В рамках текущей работы эти операции рассматриваются как предшествующая подготовка готовых данных.

Исторические материалы отражают разные состояния набора. Например, сохраненный EDA-ноутбук заканчивается выборкой из 2 937 строк, а презентация отдельно описывает `train` из 30 000 записей. Поэтому нельзя представлять все числа из этих источников как одну непрерывную, повторно подтвержденную воронку. В экспериментальной части используются фактические размеры файлов.

Согласно презентации, дополнительные характеристики извлекались из исходных описаний моделью `Meta-Llama-3.3-70B` с проверкой ответа и повторными попытками. Для животных применялся отдельный промпт. Такой способ позволяет получить согласованный JSON, но создает важную особенность постановки: часть сведений из целевого текста становится входом генератора.

Это допустимо для задачи переформулирования уже известных характеристик. Однако при первом создании объявления исходного описания еще нет. Для оценки реального пользовательского сценария параметры должны поступать из формы продавца или из независимо доступных полей. Иначе качество реконструкции текста окажется выше качества генерации по действительно доступному входу.

Исходное описание продавца не является безусловным эталоном. Оно может содержать контакты, условия доставки, повторения и рекламные обещания. Обучение на таком тексте способно закрепить поведение, запрещенное текущим промптом. Поэтому проверка качества целей необходима даже при полностью корректной структуре CSV.

Практическое следствие — отделить извлечение фактов от написания эталонного ответа. Сначала фиксируются подтвержденные характеристики, затем на их основе создается краткий текст, который проходит тот же валидатор, что и результаты модели.

3 Методы генерации описаний

3.1 Постановка мультимодальной генерации

Генератор получает изображение I , заголовок t , категорию c и параметры p . Его результат u представляет последовательность слов описания. Содержательное ограничение состоит в том, что утверждения в u должны подтверждаться доступным входом. Высокая языковая правдоподобность не заменяет выполнение этого ограничения.

Общая схема из предпроектных материалов показана на рисунке 2. На ней предусмотрено несколько фотографий, однако реализованный сервис ограничивает запрос одним изображением. Такая граница уменьшает требования к памяти и упрощает обработку [2, 4].

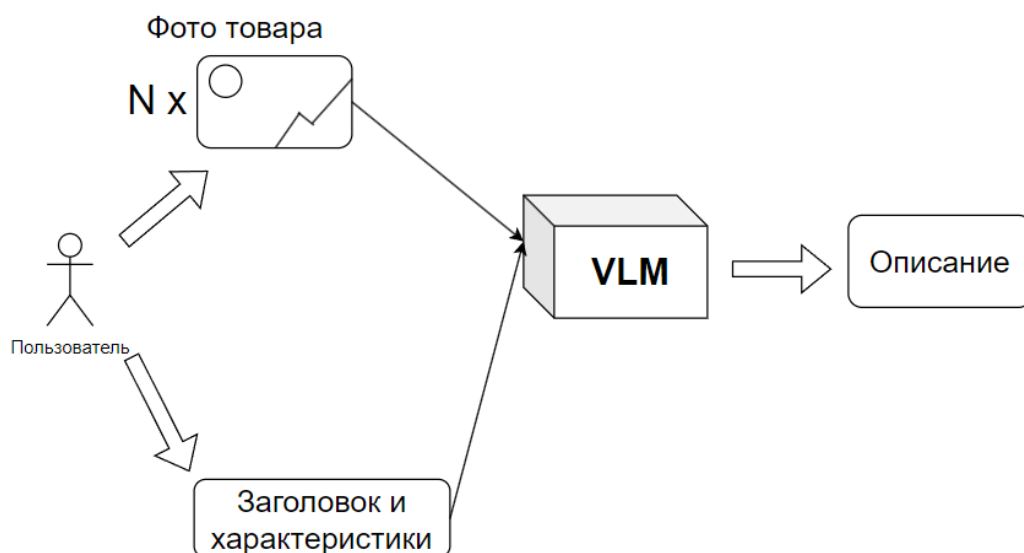


Рисунок 2 — Общая схема генерации из предпроектных материалов

Структурированные поля служат основным источником точных значений. Изображение дополняет их товарно значимыми признаками, если они уверенно различимы. Фон, поза человека или поверхность под предметом обычно не помогают покупателю и исключаются инструкцией.

В условиях неоднозначного изображения предпочтителен пропуск сомнительного свойства. Например, по фотографии можно заметить цвет корпуса, но нельзя заключить, что устройство полностью исправно. Такой подход снижает полноту описания, зато ограничивает риск ложного утверждения.

3.2 Базовые языковая и мультимодальная модели

Текстовая базовая модель в ноутбуке `inference_Vanila_LLM` — `Qwen3-4B-Instruct-2507`. Она получает заголовок, категорию и параметры без изображения. Этот вариант нужен для выяснения того, насколько структурированные данные сами по себе достаточны для составления объявления. Он также позволяет выявить случаи, когда фотография практически не меняет текст.

Мультимодальный вариант использует `Qwen3-VL-4B-Instruct`. На вход передается текстовый промпт вместе с главным изображением. Веб-сервис вызывает модель через совместимый HTTP-интерфейс `vLLM` и не зависит от внутренней реализации трансформера. В официальной карточке Qwen приводится пример сообщения с изображением и текстом [8].

Промпт сервиса содержит описание задачи, приоритет источников, ограничения использования фотографии, категорийные запреты и формат ответа. Заголовок и категория очищаются от крайних пробелов. `params` сериализуется в JSON с сохранением кириллицы и сортировкой ключей. Это обеспечивает одинаковое представление одного и того же словаря при повторных запросах.

Базовая модель не требует обучения на локальном наборе. Это снижает стоимость запуска эксперимента и дает контрольную точку для оценки полезности SFT. Вместе с тем подробная инструкция не гарантирует полного соблюдения запретов: сохраненные ответы иногда ссылаются на изображение или источник сведений.

Параметры генерации в сервисе зафиксированы отдельно от ноутбуков. По коду и `Docker Compose` используются максимум 180 новых токенов, `temperature` 0,2 и `top_p` 0,8. Токен не равен слову, поэтому лимит токенов нельзя считать реализацией правила «не более 70 слов». Для этого требуется проверка готового текста.

Сравнение текстовой и визуальной моделей должно проводиться на одинаковых объявлениях и при одинаковом содержательном контракте. Иначе различие результатов может быть вызвано промптом, длиной или способом подготовки параметров, а не вкладом фотографии.

3.3 Дообучение с низкоранговыми адаптерами

SFT обучает модель выдавать целевое описание для заданных входных данных. Его преимущество — возможность закрепить предметную лексику и желаемый стиль. Основное ограничение состоит в зависимости от качества целей: модель учится воспроизводить доступный ответ, включая его нежелательные особенности.

LoRA добавляет к выбранным весовым матрицам обучаемое изменение малого ранга, сохраняя базовые веса [9]. QLoRA сочетает такую адаптацию с квантованной базовой моделью для уменьшения требований к памяти [10]. В текстовом эксперименте применяются эти методы, а не полное обучение модели с нуля.

В ноутбуке `llm_train` задана Qwen3-4B-Instruct-2507, две эпохи, скорость обучения 0,0002, размер микропакета 1 и накопление градиента на 8 шагах. Ранг LoRA равен 16, α — 32, dropout — 0,05. Длина последовательности ограничена 768 токенами, целевой текст предварительно ограничивается 360 символами. Эти настройки характеризуют текстовую ветвь [2].

Для мультимодальной ветви сохранен ноутбук инференса базовой Qwen3-VL с LoRA-контрольной точкой `ckpt-step-3000`. Он документирует получение 100 непустых ответов без ошибок генерации. Наличие этого артефакта подтверждает использование дообученной модели, однако в доступном файле нет полного журнала ее обучения. Поэтому гиперпараметры текстовой ветви не переносятся на SFT VLM.

В инференсе SFT VLM задано до 384 новых токенов, temperature 0,3, top_p 0,7 и штраф повторений 1,05. Эти условия отличаются от сервиса. Следовательно, длинные ответы SFT нужно анализировать с учетом и обучения, и конфигурации генерации.

Снижение обучающей ошибки не доказывает повышение пригодности объявления. Если целевые тексты содержат списки и повторы, модель может точнее воспроизводить выборку и одновременно хуже выполнять требование одного краткого абзаца.

3.4 Архитектура CLIP и ruGPT

Альтернативный эксперимент реализован в `cross_attention_decision.ipynb` через `VisionEncoderDecoderModel`. Визуальный энкодер — `openai/clip-vit-base-patch32`, текстовый декодер — `sberbank-ai/ruGPT3large_based_on_gpt2`. В конфигурации декодера включены режим `decoder` и слои `cross-attention`, связывающие текст с визуальными признаками [2].

При генерации декодер последовательно предсказывает токены, учитывая текстовый контекст и представление изображения. Проекция `enc_to_dec_proj` согласует размерности скрытых представлений. Энкодер CLIP заморожен, тогда как декодер и добавленные компоненты остаются обучаемыми. По сохраненному выводу всего имеется около 1 152,8 млн параметров, из них обучаемых — 1 065,4 млн.

Эксперимент задает одну эпоху, скорость обучения 0,00004, микропакет 1, накопление градиента 64, прогрев 500 шагов и ограничение последовательности 192 токенами. Применяется оптимизатор `adamw_bnb_8bit`. Валидация и сохранение предусмотрены каждые 1 000 шагов.

В сохраненном диагностическом выводе одного примера число немаскированных целевых токенов равно нулю. Это повод отдельно проверять построение `labels`: если входной промпт занял весь лимит последовательности, для обучения по целевому описанию не остается места. Одного примера недостаточно для вывода о всем обучении, но такой случай должен выявляться автоматически.

Значительный объем обучаемого декодера и небольшой лимит контекста делают качество подготовки последовательностей особенно важным. Необходимо сохранять место под ответ, маскировать только входную часть и проверять ненулевое количество целевых токенов до запуска обучения.

Архитектурное сравнение здесь не является контролируемым исследованием одного изменения: различаются базовые модели, объем предобучения, способ объединения модальностей и режим генерации. Поэтому результат CLIP с ruGPT следует интерпретировать как результат конкретной реализации, а не доказательство общего превосходства одного механизма внимания.

3.5 Выбор модели для демонстрационного сервиса

Исследовательский набор включает пять подходов, но Docker Compose развертывает базовую Qwen3-VL-4B-Instruct. Подключение LoRA-адаптера к runtime в текущей конфигурации не задано. Поэтому демонстрационный интерфейс и экспериментальная SFT VLM представляют разные конфигурации и не должны называться одной моделью.

Выбор базовой VLM для MVP обоснован простотой развертывания, наличием штатного мультимодального интерфейса и лучшим соблюдением текущих формальных правил среди доступных сохраненных ответов. Это инженерное решение для демонстрации. Оно не отменяет более высокий балл SFT VLM, указанный на слайде сравнения при другом протоколе.

Промпт и модель образуют единый вариант генератора. При сравнении необходимо фиксировать версию инструкции, имя весов, адаптер, параметры декодирования и постобработку. Изменение любого из этих элементов может повлиять на результат даже при неизменном названии базовой модели.

В клиенте vLLM предусмотрена постобработка: пробельные символы сворачиваются, начальный префикс «Описание:» удаляется, предложения с запрещенными ссылками на источник отбрасываются. Такая операция помогает получить единый абзац, но может удалить и полезные факты. После удаления всех предложений возможен пустой результат, который требует дополнительной проверки.

Следующий шаг — сравнить модели после одинаковой обработки при одинаковом ограничении длины. При этом следует сохранять как исходный, так и итоговый ответ. Это позволит различать качество генератора и эффект фильтра, а также оценить потерю информации из-за удаления предложений.

Подмена текста шаблоном, повторная генерация и ручное исправление могут быть резервными сценариями, но в данном отчете они рассматриваются как направления развития. Наличие таких механизмов в реализованном MVP не предполагается без подтверждения кодом.

4 Реализация программной системы

4.1 Компоненты и взаимодействие

Система разделена на клиентское приложение, API и модельный runtime. Такое разделение позволяет изменять интерфейс независимо от способа обслуживания модели. Состав контейнеров и маршруты описаны в таблице 4 [2].

Таблица 4 — Компоненты демонстрационного сервиса

Компонент	Технология	Ответственность
frontend	React, Vite, nginx	Форма, предпросмотр, отображение и копирование
api	FastAPI, Python	Проверка входа, промпт, обращение к модели
qwen-vllm	vLLM, Qwen3-VL	Мультимодальный инференс на GPU

Браузер обращается к frontend на порту 5173. nginx передает запросы генерации и состояния во внутренний api:8080. API доступен с хоста на порту 8081, а модельный сервер — на 8000. Backend обращается к адресу qwen-vllm:8000/v1 внутри контейнерной сети.

Маршрут POST /generate-description получает multipart/form-data. После разбора полей сервис строит промпт и data URL изображения, затем клиент отправляет мультимодальное сообщение на /chat/completions. Из первого варианта ответа извлекается content, выполняется очистка и возвращается JSON с полем description.

Интерфейс ProductDescriptionModel определяет метод generate. Благодаря этому логика подготовки объявления отделена от транспортного клиента и может проверяться с тестовой заглушкой. Settings хранит параметры модели, лимиты и адрес runtime; загрузка окружения сосредоточена в одном модуле.

GET /health подтверждает, что API работает. GET /ready обращается к списку моделей и проверяет наличие нужного идентификатора. Такое разделение важно при первом запуске: HTTP-приложение может уже отвечать, пока веса еще загружаются.

4.2 Валидация и обработка ошибок

Backend удаляет крайние пробелы заголовка и категории, после чего проверяет их непустоту. params должен быть JSON-объектом: массив, число и некорректный JSON отклоняются. Пустая строка параметров интерпретируется как пустой словарь, что позволяет описывать товар с минимальным набором сведений.

Максимальный размер изображения в текущих настройках составляет 10 485 760 байт. Поддерживаются JPEG, PNG и WEBP. Модуль image_utils проверяет известные сигнатуры; при отсутствии распознанной сигнатуры он допускает выбор по заявленному Content-Type. Следовательно, это ограниченная проверка формата, а не полная проверка декодируемости изображения.

Ошибки данных преобразуются в HTTP 422, а ошибки модельного клиента — в HTTP 503. Клиент обрабатывает сетевую недоступность, ошибочный HTTP-статус runtime, некорректный JSON и неожиданную структуру ответа. Контракт ошибок приведен в таблице 5.

Таблица 5 — Поведение API при типовых ситуациях

Ситуация	Ответ	Смысл для клиента
Корректный запрос	200	Поле description с результатом
Некорректные поля	422	Исправить данные запроса
Недоступность runtime	503	Повторить после восстановления
API запущен	health: ok	Процесс отвечает
Модель обнаружена	ready: true	Runtime сообщает нужную модель

Граница доверия проходит и через содержимое входа. Свободный заголовок может содержать инструкцию, конфликтующую с системными ограничениями. Текущий промпт задает правила, но отдельный механизм защиты от таких вмешательств не реализован.

Для развития сервиса полезны декодирование изображения до инференса, ограничение разрешения и итоговая проверка непустого результата. Эти изменения следует проверять отдельно, поскольку формально допустимый запрос может создавать чрезмерную нагрузку или возвращать непригодный текст.

4.3 Пользовательский интерфейс

Frontend реализован на React и TypeScript. Пользователь выбирает или перетаскивает изображение, заполняет заголовок, категорию и JSON-параметры. Состояние формы включает готовность runtime, ход запроса, текст результата и ошибку. Снимок демонстрации из презентации приведен на рисунке 3.

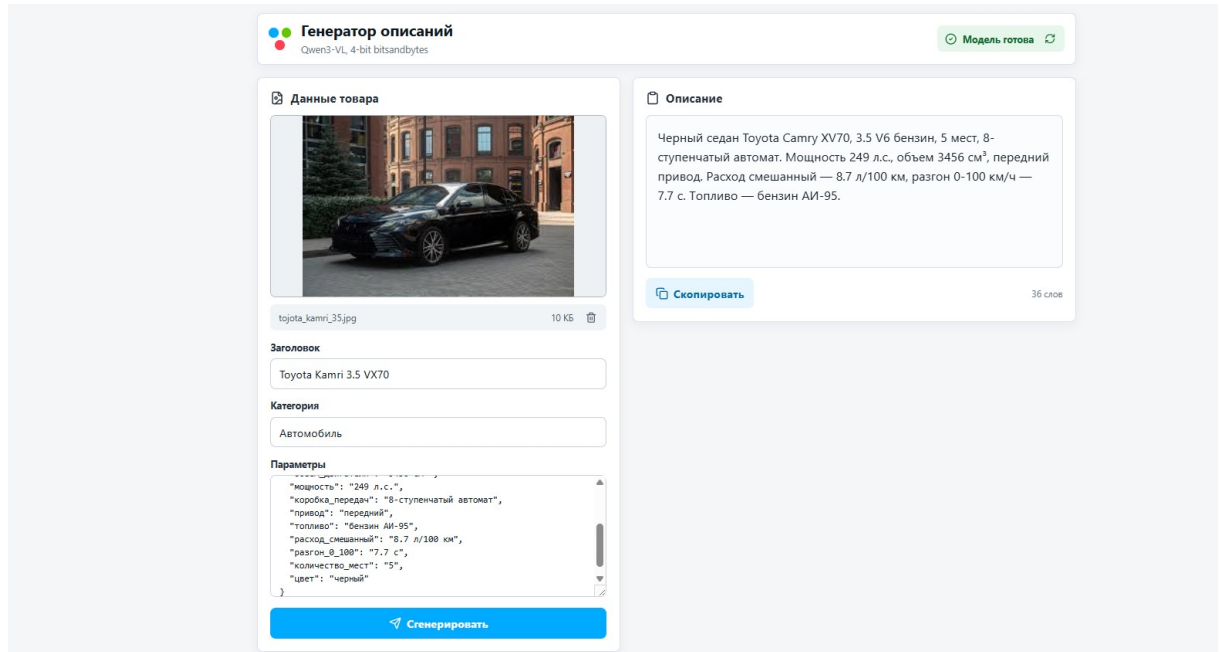


Рисунок 3 — Интерфейс генератора описаний из презентации проекта

Кнопка отправки доступна, когда заполнены обязательные поля, выбрано изображение, params успешно разобран и запрос еще не выполняется. Индикатор готовности модели отображается отдельно. По текущему коду он не является обязательным условием canSubmit, поэтому окончательное сообщение о недоступности может прийти от backend.

Предпросмотр использует временный URL браузера; при смене изображения он освобождается. После успешного запроса показывается описание и доступно копирование в буфер обмена. При ошибке пользователь получает сообщение без необходимости анализировать сетевой ответ вручную.

На рисунке показан частный пример с автомобилем. Он иллюстрирует пользовательский сценарий, но не заменяет проверку достоверности характеристик. Для массовой оценки используются фиксированные CSV-ответы и единые критерии, а не отдельный удачный снимок интерфейса.

4.4 Развертывание и программные проверки

Контейнерная конфигурация содержит frontend, api и qwen-vllm. Для весов используется отдельный том hf-cache, позволяющий не скачивать модель заново при каждом запуске. Модель работает в режиме bitsandbytes 4-bit с dtype float16; это режим runtime, а не доказательство наличия отдельного GPTQ- или AWQ-чекпоинта.

В Docker Compose установлены max-model-len 2048, max-num-seqs 1 и доля использования памяти GPU 0,90. Разрешено одно изображение на запрос. Ограничение одновременно обслуживаемых последовательностей влияет на очередь: увеличение числа клиентов само по себе не увеличивает пропускную способность модели.

Проверки проекта охватывают построение промпта, JSON-параметры, преобразование изображения, работу сервиса с заглушкой модели, транспортный запрос и очистку ответа. Отдельные тесты проверяют вспомогательные функции оценки на датасете и нагрузочного сценария. Во frontend предусмотрены проверки формы, сообщений об ошибках и multipart-контракта.

При подготовке отчета выполнены существующие Python-тесты через unittest: обнаружено 24 теста, успешно выполнен 21, три API-теста пропущены из-за отсутствующих тестовых зависимостей FastAPI. Запуск frontend-тестов и полный GPU-прогон в этой проверке не проводились. Поэтому результат подтверждает только охваченные локальные функции.

Тест с подмененным клиентом полезен для проверки формата запроса, но не измеряет качество Qwen и не подтверждает работу CUDA. Полная приемка развертывания должна включать загрузку весов, проверку /ready и реальный запрос с изображением. Краткая последовательность запуска и фиксации результата приведена в приложении А.

Перед длительным экспериментом необходимо сохранять конфигурацию контейнеров, имя модели, параметры генерации и версию промпта. Без этого одинаковый HTTP-контракт может скрывать разные условия инференса, а сравнение результатов окажется неоднозначным.

5 Методика и результаты оценки

5.1 Двухэтапный контроль качества

Модуль LLM-as-a-Judge разделяет проверку формата и содержательную оценку. Сначала `ad_validator` проверяет явные нарушения, затем допущенный ответ оценивается мультимодальным судьей. Если программная проверка не пройдена, запрос к судье не выполняется [3]. Такой порядок сокращает число дорогих вызовов.

Правила проверяют пустой ответ, перенос строки, превышение 70 слов, префикс «Описание:», некоторые виды разметки и списков, приветствия, ссылки на фото, URL, адреса почты, телефоны и контакты мессенджеров. Важно учитывать точную реализацию: любой символ перевода строки трактуется как нарушение одного абзаца, а поиск слов выполняется регулярным выражением.

Судья получает исходные поля, изображение и сгенерированный текст. Описание продавца используется только как слабый ориентир и не легализует контакты или обещания из исходного текста. Оценка строится по четырем рубрикам: опора на источники, релевантность визуальных признаков, соблюдение задания и качество языка.

Каждая рубрика имеет диапазон от 0 до 1. Общий балл одного ответа в коде рассчитывается по формуле (1), затем округляется до десятых.

$$s = (g + v + c + l) / 4 \quad (1)$$

где g — оценка опоры на источники;
 v — оценка использования изображения;
 c — оценка соблюдения задания;
 l — оценка качества языка.

Применение LLM в роли оценщика позволяет проверять содержание без единственного эталонного текста, но требует контроля смещений и согласованности с человеком [11]. Поэтому высокая оценка судьи рассматривается как диагностический сигнал, а не гарантия истинности каждого утверждения.

5.2 Ансамбль судей и сопоставимость результатов

В конфигурации judge указаны gpt-5.2, claude-4-6-opus и grok-4. Это идентификаторы, используемые проектным клиентом. Пакетная оценка строит задания для каждой строки, колонки ответа и модели-судьи. При 100 объявлениях и одном генераторе возможны 300 модельных оценок, если все строки проходят предварительные проверки и отсутствуют ошибки.

Средний балл нужно сопровождать числом допущенных примеров. Если один генератор проходит фильтр только на простых объектах, среднее по оставшимся ответам не характеризует весь набор. Для сравнения следует одновременно сообщать долю прохождения правил, число ошибок и качество допущенных ответов.

Сохраненная презентация описывает более мягкую рубрику: некоторые приветствия и оценочные слова не штрафуются. Текущий код, напротив, запрещает приветствия и требует подтверждения характеристик. Следовательно, средние баллы из презентации и новая проверка ad_validator отвечают на разные вопросы.

Обнаружено и различие CSV-контракта. В файле CLIP с ruGPT колонка результата называется pred_description, а список автоматически распознаваемых колонок judge этого имени не содержит. При неосторожном запуске может быть выбран description, то есть исходный текст продавца. Для локальной проверки в настоящем отчете pred_description передавался валидатору явно.

Полный повторный прогон VLM-судей не выполнялся: в комплекте нет каталога изображений и сохраненных поэлементных оценок, подтверждающих все числа презентации. Поэтому далее отдельно представлены опубликованная проектная сводка и заново рассчитанные формальные показатели.

Для будущего воспроизводимого сравнения требуется зафиксировать единый промпт судьи, один список объектов и явное соответствие колонок моделям. Дополнительно полезно сохранять комментарии судей и проверять спорные случаи вручную, скрыв от проверяющего название генератора.

5.3 Сравнение подходов по материалам проекта

На слайде 10 презентации приведены средние оценки пяти вариантов. Они сведены в таблицу 6 без изменения соответствия названия и числа [1]. Эти

значения являются сообщенными результатами проекта; поэлементные протоколы для их независимого пересчета в доступных файлах отсутствуют.

Таблица 6 — Оценки генераторов на слайде сравнения

Вариант	Средняя оценка
Текстовая LLM без SFT	0,54
Текстовая LLM после SFT	0,70
CLIP и ruGPT с cross-attention	0,67
Базовая VLM	0,69
SFT VLM	0,78

По этой таблице SFT VLM имеет наибольшее указанное значение. Разница относительно базовой VLM составляет 0,09, а текстовое дообучение сопровождается ростом с 0,54 до 0,70. Эти арифметические различия сами по себе не показывают статистическую значимость, поскольку нет распределения по объектам и согласованности судей.

В презентации имеются внутренние несоответствия. Заголовок следующего слайда утверждает снижение метрик после SFT, хотя рядом показан рост с 0,69 до 0,78. Итоговый слайд приписывает 0,78 базовой VLM. Поэтому нельзя одновременно использовать эти утверждения как согласованный вывод. Для таблицы принят конкретный слайд с сопоставлением пяти подходов.

Допустимый вывод состоит в том, что мультимодальное дообучение перспективно по сообщенной содержательной оценке, а окончательное ранжирование требует согласованного повторного эксперимента. Причины возможного ухудшения отдельных свойств текста обсуждаются как гипотезы, а не как доказанное падение общего балла.

Для практического выбора дополнительно важны длина, наличие контактов и точность характеристик. Модель с высоким средним баллом может быть непригодна для текущего интерфейса, если стабильно нарушает жесткий формат. Поэтому следующая проверка рассматривает сохраненные ответы по единым программным правилам.

5.4 Повторная проверка сохраненных ответов

К каждой из трех доступных колонок применен текущий `validate_description` без изменения правил. Во всех файлах по 100 непустых ответов. Для CLIP с ruGPT явно использован `pred_description`. Результат представлен на рисунке 4 и в таблице 7.

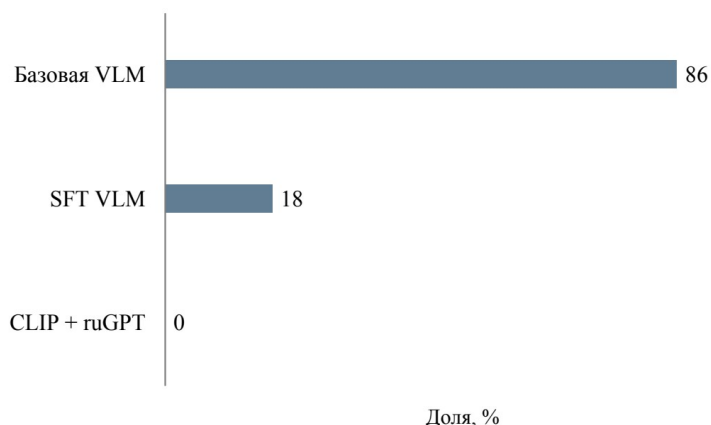


Рисунок 4 — Прохождение текущих программных правил сохраненными ответами

Таблица 7 — Основные нарушения формата

Показатель	Базовая VLM	SFT VLM	CLIP и ruGPT
Прошли все правила	86	18	0
Есть перенос строки	0	78	99
Более 70 слов по валидатору	0	65	96
Есть разметка или список	0	10	11
Ссылка на источник или фото	14	1	5

Один ответ может иметь несколько нарушений, поэтому строки таблицы не суммируются в 100. Для CLIP с ruGPT дополнительно выявлены 53 ответа с телефонным паттерном, 27 с URL и 12 с email. Это срабатывания регулярных выражений; они требуют проверки возможных ложных совпадений с артикулами.

Результат 86 % у базовой VLM означает прохождение формальных правил, а не 86 % фактически верных объявлений. Проверка не видит фотографию и не определяет истинность утверждений. Аналогично нулевое прохождение CLIP с ruGPT означает несовместимость сохраненных ответов с текущим строгим форматом, а не отсутствие в них полезных сведений.

5.5 Анализ конкретного ответа и ограничений

Первый объект общего проверочного набора — красный ремешок Garmin Quickfit шириной 26 мм. В params указаны бренд NNNB, модель For Garmin Fenix 5/5X, силикон, красный цвет и состояние «Новый, без бирок». Этот пример позволяет сравнить тексты при одинаковых входных полях [3].

Базовая VLM выдает краткий абзац: «Ремешок для Garmin Quickfit 26 мм, красный, силиконовый, унисекс. Бренд NNNB, модель для Garmin Fenix 5/5X. Новый, без бирок. Ширина 26 мм». Значения сохранены, но ширина повторяется. Это пример формально допустимого текста с редакционным недостатком.

SFT VLM формирует многострочный перечень характеристик, многократно повторяет материал, пол и тип товара. В тексте появляется происхождение из Китая, которого нет в приведенном словаре. Без отдельного подтверждения по изображению такое дополнение нельзя считать обоснованным. Повторы и структура списка уже достаточны для отклонения по текущему формату.

CLIP с ruGPT пишет о ширине 25 мм вместо 26 мм, добавляет гарантию, возврат и доставку. Ошибка размера подтверждается прямым сравнением с заголовком и params и не требует визуального судьи. Этот случай показывает, почему общая связность русского текста недостаточна для задачи объявления.

На уровне всего набора базовая модель лучше соблюдает ограничения длины и структуры, а SFT склонна воспроизводить развернутые продавцовские шаблоны. Это наблюдение согласуется с гипотезой влияния целевых описаний, однако не устанавливает причинность: различаются промпты, параметры генерации и обработка результатов.

Главные ограничения оценки — малый размер valid, отсутствие одной из категорий, неодинаковые конфигурации генераторов и отсутствие полного протокола исторического судейства. Для проверки фактической точности требуется ручная разметка утверждений с привязкой к полю или фрагменту изображения. Без нее формальные и модельные оценки остаются взаимодополняющими, но неполными измерениями.

6 Эксплуатационные характеристики и развитие

6.1 Производительность и границы нагрузки

В презентации приведены результаты нагрузочных измерений демонстрационного сервиса. Они представлены в таблице 8 как сведения исходного проекта, а не повторный замер при подготовке отчета. Полные JSONL-журналы измерений в доступном комплекте отсутствуют [1, 2].

Таблица 8 — Сообщенные характеристики демонстрационного сервиса

Показатель	Значение и условие
Видеокарта	RTX 3060, 12 ГБ VRAM
Пиковое потребление VRAM	До 11 из 12 ГБ на стресс-прогоне
Задержка p95	3,4–5,7 с при 2–4 одновременных запросах
Пропускная способность	Около 0,75 запроса/с
Стресс-сценарий	64 запроса без ошибок при timeout 300 с
Пиковая мощность GPU	Около 170 Вт

Число одновременно отправленных запросов не равно числу одновременно вычисляемых последовательностей. При `max-num-seqs 1` часть запросов ожидает в очереди. Поэтому успешная обработка 64 запросов при большом тайм-ауте не означает комфортную интерактивную работу 64 пользователей.

Показатель p95 полезнее одного среднего значения, поскольку отражает задержку у более медленных запросов. При дальнейшем тестировании нужно сохранять число запросов, длительность прогрева, размер изображений, длину ответов и долю ошибок. Без этих условий результаты разных запусков трудно сопоставлять.

В README рекомендован рабочий диапазон 2–4 параллельных запроса. Его можно использовать как исходную конфигурацию демонстрации, но не как гарантированную производительность. Расширение диапазона требует отдельного подбора `batching`, лимита последовательностей и потребления памяти с повторным измерением качества ответа.

6.2 Практический эффект и дальнейшая работа

Ожидаемый эффект MVP — уменьшение ручной работы над черновиком и более полное представление подтвержденных характеристик. Фактическое влияние на заказы в проекте не измерялось. Для проверки необходим A/B-тест с учетом принятия текста, объема правок и нежелательных последствий.

В исходном бизнес-обосновании показатель Edit Rate смешан с принятием ответа. Для эксперимента их следует разделить: Acceptance Rate — доля использованных генераций, Edit Rate — доля ответов, которые продавец изменил. Рост первой метрики обычно желателен, а смысл второй зависит от величины и причины правок.

Денежная оценка в предпроектном документе требует уточнения. При условных 10 млн случаев, конверсии 5 %, чеке 5 000 руб. и относительном приросте 1,7 % получается 42,5 млн руб. дополнительного оборота за период. Это сценарный расчет, а не выручка платформы и не результат MVP: доля платформы и затраты не заданы.

Первое направление развития — переразметка целей под единый краткий формат и независимая проверка характеристик. Второе — обучение на предпочтениях, например DPO, где сравниваются выбранный и отклоненный ответы [12]. Наличие таких методов в плане не означает их реализацию в текущем коде.

Предпроектные материалы также содержат вариант с отдельными визуальными ветвями и адаптерами, показанный на рисунке 5. Он рассматривается как исследовательская идея, а не фактическая архитектура развернутого Qwen-сервиса.

6.3 План развития архитектуры

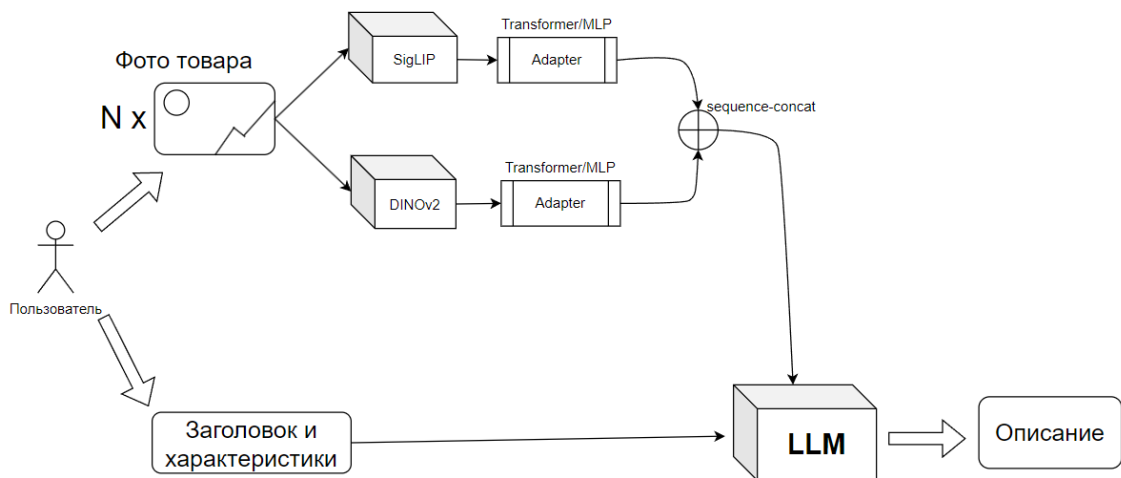


Рисунок 5 — Предпроектный вариант объединения визуальных признаков

Схема предусматривает обработку фотографий двумя визуальными ветвями, обозначенными SigLIP и DINOv2, преобразование признаков адаптерами и объединение перед подачей в языковую модель. Ее исследовательский смысл — проверить, дает ли объединение представлений дополнительные сведения о товаре. Преимущество такой схемы в проекте пока не измерено.

Перед реализацией необходимо определить формат визуальных токенов, способ согласования размерностей и допустимое число изображений. Затем следует отдельно сравнить одну и две ветви на одинаковом наборе объектов. Иначе выигрыш от дополнительной фотографии будет смешан с эффектом другой архитектуры.

Приоритетным остается улучшение качества данных и протокола оценки. Дополнительные компоненты увеличивают сложность обучения и эксплуатации, поэтому их внедрение оправдано только при измеримом улучшении достоверности или полноты описания. Помимо среднего балла потребуется сравнить задержку, потребление памяти и частоту нарушений.

Практический порядок развития следующий: согласовать единый краткий формат, подготовить проверенные целевые ответы, повторить SFT и только затем расширять визуальный вход. Для каждого этапа сохраняется предыдущая конфигурация как контрольная. Такой порядок позволяет связывать изменение результата с конкретным изменением системы.

ЗАКЛЮЧЕНИЕ

В курсовой работе рассмотрена и реализована задача генерации русскоязычных описаний объявлений Авито по одной фотографии, заголовку, категории и характеристикам товара. Подготовлен демонстрационный сервис, объединяющий React-интерфейс, FastAPI и vLLM с Qwen3-VL-4B-Instruct. Система принимает данные, формирует запрос к модели и возвращает описание для проверки пользователем.

Проверены готовые табличные данные: 30 000 обучающих объектов, 100 проверочных и 2 937 объектов дополнительной выборки. Пересечения train и valid по item_id не обнаружены. Неравномерность категорий и отсутствие животных в valid ограничивают перенос общих результатов на весь ассортимент.

Исследовательская часть охватывает текстовые и мультимодальные модели до и после SFT, а также CLIP с ruGPT и cross-attention. На слайде сравнения максимальная сообщенная оценка составляет 0,78 для SFT VLM. Из-за противоречивых итоговых подписей и отсутствия поэлементного протокола этот результат не трактуется как окончательно подтвержденное ранжирование.

Повторная программная проверка сохраненных ответов дала 86 % прохождения текущих правил у базовой VLM, 18 % у SFT VLM и 0 % у CLIP с ruGPT. Эти показатели демонстрируют различия в соблюдении формата и объясняют практический выбор базовой модели для MVP. Они не заменяют оценку фактической достоверности.

Основная цель — построение прототипа и исследование вариантов генерации — достигнута. Подтверждены структура данных, наличие программных компонентов и результаты локальных проверок. Промышленная интеграция, пользовательский А/В-тест и повторное полное судейство остаются за пределами выполненной проверки.

Продолжение работы целесообразно начать с единого контракта данных и оценки, очистки целевых описаний, расширения проверочного набора и ручного аудита фактов. После этого можно обоснованно сравнить дообучение, обучение на предпочтениях и добавление нескольких изображений, не смешивая улучшение содержания с изменением правил проверки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ханенко Д., Хаялиев Р. Генерация описаний для объявлений : презентация проекта. — 2026. — 15 слайдов. — Файл: AAA_presentation.pptx.
2. ДоLMатинцы. Генерация описаний для объявлений : исходный код, ноутбуки и документация проекта. — 2026. — Локальный репозиторий: aaa-description_generation. — Используются notebooks, service/app, service/tests, service/frontend и docker-compose.yml.
3. LLM-as-a-Judge : исходный код, промпты, документация и сохраненные ответы моделей. — 2026. — Локальный репозиторий: LLM_as_a_judge. — Используются ad_validator.py, ad_scorer.py, evaluation_models.py, docs и vals.
4. Оценка проекта до взятия в работу : предпроектное обоснование и схемы решения. — 2026. — Файл: бизнесчасть_проблема_актуальность_аналоги.docx.
5. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Москва : Стандартинформ, 2017. — 28 с.
6. Кляхандлер С. Как мы разработали LLM-модель, которая генерирует описания товаров для пользователей Авито // Хабр. Блог AvitoTech. — 02.05.2025. — URL: <https://habr.com/ru/companies/avito/articles/906504/> (дата обращения: 19.09.2026).
7. Кляхандлер С. Как DS-инженеры совершенствуют автогенерацию описаний и пополняют индекс объявлений Авито // Хабр. Блог AvitoTech. — 19.09.2025. — URL: <https://habr.com/ru/companies/avito/articles/948626/> (дата обращения: 19.09.2026).

8. Qwen Team. Qwen3-VL-4B-Instruct : карточка модели. — URL: <https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct> (дата обращения: 19.09.2026).
9. Hu E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models. — 2021. — arXiv:2106.09685. — URL: <https://arxiv.org/abs/2106.09685> (дата обращения: 19.09.2026).
10. Dettmers T. et al. QLoRA: Efficient Finetuning of Quantized LLMs. — 2023. — arXiv:2305.14314. — URL: <https://arxiv.org/abs/2305.14314> (дата обращения: 19.09.2026).
11. Zheng L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. — 2023. — arXiv:2306.05685. — URL: <https://arxiv.org/abs/2306.05685> (дата обращения: 19.09.2026).
12. Rafailov R. et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. — 2023. — arXiv:2305.18290. — URL: <https://arxiv.org/abs/2305.18290> (дата обращения: 19.09.2026).

ПРИЛОЖЕНИЕ А

(справочное)

Порядок запуска и фиксации эксперимента

Перед запуском должны быть доступны Docker с GPU support, драйвер NVIDIA и место для весов модели. Команда выполняется из каталога service, где находится docker-compose.yml. В таблице A.1 приведены операции для локального демонстрационного режима.

Таблица A.1 — Операции запуска и проверки

Операция	Команда или адрес
Запуск контейнеров	<code>docker compose up --build</code>
Проверка API	<code>http://localhost:8081/health</code>
Проверка модели	<code>http://localhost:8081/ready</code>
Открытие интерфейса	<code>http://localhost:5173</code>
Просмотр runtime	<code>docker compose logs --tail 200 qwen-vllm</code>
Остановка	<code>docker compose stop</code>

Для воспроизводимого эксперимента сохраняются идентификаторы объявлений, версия промпта, модель и адаптер, параметры генерации, исходный и очищенный ответы, ошибки и время выполнения. Поэлементные оценки судей сохраняются отдельно от агрегатов.

Перед оцениванием необходимо явно сопоставить колонку ответа генератору. Для CLIP с ruGPT используется `pred_description`, для базовой VLM — `qwen_base_response`, для SFT VLM — `qwen_sft_response`. Поле `description` остается исходным текстом продавца.

При повторной проверке правил следует использовать одинаковую версию `ad_validator` для всех вариантов. Полный прогон судей дополнительно требует фотографий, настроенного доступа к моделям и фиксированной рубрики. Секреты доступа и содержимое служебных переменных в отчет не включаются.